Atividade da Aula 11 — Construir uma fatia e testá-la

Valor: 0,5 (duas das quatro entregas que compõem o 1,0 de atividades de aula da Unidade 3)
Formato: produção em grupo (3 a 5 pessoas), **entrega individual** no Teams
Prazo: 05/11, até 23h59

> É **uma atividade só**, em duas partes que seguem a mesma história: na **Parte A** o grupo corta uma história e constrói a primeira fatia; na **Parte B** testa o que construiu. Os testes são os "testes derivados dos critérios de aceite, incluindo casos limite e de erro" que o **Trabalho 3** pede — e a rede de proteção da **Aula 12**, quando o banco for trocado. A Unidade 3 termina em **03/12**, com a Prova 3 e a entrega do Trabalho 3 (branch `entrega-3`). O que vocês fazem aqui é a primeira das **três histórias novas** que o Trabalho 3 pede.

Escolham uma história que tenha **regra de negócio** (algo que o sistema recusa ou decide), não só uma tela: é nela que os testes e a inversão da Parte B fazem sentido.

Parte A — A definição de pronto e a primeira fatia

Duas coisas no repositório do grupo, as duas entradas por **PR revisado**, como qualquer mudança desde a Aula 6.

A1. A definição de pronto, no `README.md`

Uma seção com exatamente este nome — a correção procura por ele:

```
## Definição de pronto

Pronto, no nosso grupo, é:

- [ ] <item>  — onde se confere: <aba Checks · aba Reviews · diff do PR · tela do celular · ...>
- [ ] <item>  — onde se confere: <...>
```

- **4 itens ou mais**, e cada item com **onde se confere** — o lugar em que outra pessoa o verifica sem perguntar ao autor.
- Dois itens são obrigatórios, porque já são regra da disciplina: **CI verde no PR** e **aprovado por um integrante que não é o autor**.
- Os demais são do grupo. Itens que ninguém confere com sim ou não ("código limpo", "bem testado", "sem bugs") não contam.
- A seção não existe no template: o grupo a cria. Recomendado, sem valer ponto: levar os mesmos itens para o checklist de `.github/pull_request_template.md`.

A2. O PR da primeira fatia de uma história

A próxima história priorizada do backlog de `docs/analise.md`, **cortada em fatias**, e o PR da **primeira** delas — **aberto ou integrado**. A descrição do PR traz:

- **Título** que nomeia o incremento, como alguém o usa (não "PR 1", não "Trabalho 3").
- **`História / decisão relacionada`** preenchido: a história, o critério de aceite da fatia e, se houver, o ADR.
- **As fatias da história**, uma por linha, cada uma com o seu critério em uma frase — a primeira é a deste PR.

- **Como verificar:** os passos que o revisor repete.
- `Uso de IA` preenchido: o que a IA gerou, o que ficou e o que vocês mudaram, e por quê — ou "não usamos".

E o diff do PR contém **pelo menos um teste novo** para o critério da fatia: um `it` com o nome do critério, não um `it.todo`.

Recomendado, sem valer ponto — escrevam o critério da fatia em Dado, Quando e Então. É nessa forma que ele vira teste. Exemplo do formato: *Dado que o doador publicou uma doação, quando ele a cancela, então ela não aparece na lista de disponíveis.*

Parte B — O PR de testes

Um **segundo PR**, também revisado, com os testes da **história da Parte A**. O critério de aceite de cada fatia, que está na descrição do PR da fatia e em `docs/analise.md`, é a fonte dos testes. Os testes vão em `tests/` e passam pela API, com `supertest` e `vitest`, como o `tests/doacoes.test.js` do template.

O teste que o PR da fatia já trouxe **não conta como novo**: ele é o ponto de partida, e pode ser reforçado.

B1. Os testes dos critérios

3 testes novos ou mais dos critérios de aceite da história. Cada um é um `it` — não um `it.todo` — e cumpre:

- o **nome do teste é o critério ou a regra** (não "teste 1", não "deve funcionar");
- o **Dado** prepara o estado pela própria API, o **Quando** é a ação e o **Então** confere o **valor** da resposta, não só o status;
- o Dado, o Quando e o Então aparecem à vista (por exemplo, em comentários).

Se só a primeira fatia está na `main`, os testes cobrem **o que o critério dela promete** (o efeito, o estado depois, o que o usuário vê) e **a regra de negócio** que ela implementa. Se o grupo já integrou outras fatias, os critérios delas entram também. Se um critério ainda não tem código, o teste vem primeiro e **fica vermelho**: implementem a fatia no mesmo PR, em commit à parte.

B2. Dois casos limite ou de erro

2 testes novos ou mais do que o critério original **não previa**, cada um com o nome da regra e conferindo a **recusa**, a **mensagem** ou o **vazio** — não só o `200`. As quatro perguntas ajudam: *e se faltar algo, e se vier vazio, e se repetir, e se não existir?*

Se um desses testes ficar vermelho, o produto tem um buraco: corrijam-no **no mesmo PR, em commit à parte**, e digam isso na descrição.

B3. A inversão de uma regra

Escolham **uma regra que a fatia da Parte A implementa** e que tenha teste, e façam, nesta ordem:

1. **Commitar** o que já funciona.
2. **Inverter a regra** no código (por exemplo, apagar a validação que a protege).
3. **Rodar a suíte inteira** e **contar os vermelhos**, sem corrigir nada antes da contagem.
4. **Desfazer** a inversão.

Se nenhum teste ficou vermelho, esse teste é decorativo: reforcem-no e repitam o experimento. O registro vai na descrição do PR de testes, numa seção com exatamente este nome — a correção procura por ele:

```
## Inversão de regra

- Regra invertida: <qual regra, em que arquivo, o que foi alterado>
- Resultado: <N testes vermelhos: os nomes> — ou "nenhum", e o que o grupo fez depois
- Inversão desfeita: sim
```

A inversão é um experimento e **não entra no PR**: o código volta ao que era, e o CI do PR fica verde.

B4. A ligação entre as partes e o Uso de IA

A descrição do PR de testes traz, além da inversão:

```
## História testada

- PR da fatia: <url>
- Critérios cobertos: <um por linha, com o nome do teste que o confere>
```

E a seção `## Uso de IA` preenchida: o que a IA gerou, o que ficou, o que vocês mudaram e por quê — ou "não usamos".

Além dos entregáveis

- **Cada integrante faz ao menos um commit** na atividade — ou aparece como `Co-authored-by:` no commit do par. O histórico do repositório é o que comprova participação.
- **O PostgreSQL no ar**, pelo caminho escolhido no ADR da migração, e **a suíte verde na `main`. Não valem ponto**, mas a Aula 12 troca o código para o banco novo com esta suíte como prova: quem chegar lá sem o banco respondendo, ou sem suíte verde, não executa a migração em sala nem tem como mostrar que o comportamento ficou.
- **CI verde** e a `main` **protegida**.

Como entregar

No Teams, na atividade da Aula 11, cole exatamente estas quatro linhas:

```
Repositório: <url do repositório do grupo>
PR da fatia: <url do PR da primeira fatia>
PR dos testes: <url do PR de testes>
Meu commit: <url de um commit seu, feito durante a atividade>
```

Os PRs podem estar **abertos ou integrados**. A definição de pronto é conferida no `README.md` da `main` no prazo. A produção é do grupo; a entrega é individual, porque a nota é individual.

Como é corrigido

Binário: entregou no prazo e no formato, pontuou os 0,5. Não há nota parcial. A conferência é de presença e de forma, e a lista abaixo é exaustiva; a qualidade é avaliada no Trabalho 3.

Perde o ponto quem: entrega fora do prazo · não tem `## Definição de pronto` no `README.md` da `main` · tem menos de **4 itens**, ou algum item **sem onde se confere**, ou não inclui CI verde e revisão por outro integrante · manda o link do repositório sem os links dos dois PRs · entrega um PR da fatia sem `História / decisão`, sem a lista das fatias, sem `Uso de IA` preenchido ou **sem teste novo** do critério (`it.todo` não conta) · entrega um PR de testes com **menos de 3 testes novos de critério** · com **menos de 2 testes novos de limite ou de erro** · sem a seção `## Inversão de regra`, ou com ela **sem a regra invertida**, **sem o resultado** (os nomes dos testes vermelhos, ou "nenhum" e o que o grupo fez) ou **sem "inversão desfeita"** · sem a seção `## História testada`, ou com ela **sem o link do PR da fatia** · entrega um PR de testes com o **CI vermelho** ou com o `Uso de IA` sem preencher · não tem commit próprio na atividade (a conferência é individual: quem não tem commit seu perde o ponto).

A qualidade é avaliada depois, e vale muito mais: "três histórias novas integradas por PR revisado", "testes cobrem os critérios de aceite das histórias implementadas" e "commits e PRs mostram evolução incremental e participação de todos" são critérios de aceite do **Trabalho 3** (2,0 do artefato). Na **defesa individual** (1,0) em **03/12**, cada integrante faz uma pequena modificação ao vivo e responde sobre uma parte que não escreveu — e saber o que cada teste protege é o caminho até ela.

Nível de IA nesta atividade: IA como colaboradora

Pode: escrever o rascunho do código e dos testes da fatia, sugerir como cortar a história e casos limite, explicar uma mensagem de erro do `vitest`, revisar uma mensagem de commit.

Antes de aceitar o código gerado, as seis verificações: (1) entendo cada linha? (2) respeita as camadas — regra em `src/doacoes.js`, SQL em `src/repositorio.js`, só o `src/db.js` conhece o banco? (3) o SQL usa marcadores, sem valor colado na string? (4) inventou função, coluna, rota ou pacote? (5) vaza dado pessoal em log, resposta ou erro? (6) o teste pegaria o erro?

Antes de aceitar os testes gerados: (1) para cada `expect`, **o que precisa quebrar para ele falhar?** (2) o Dado prepara o estado, ou o teste depende de um dado que já estava lá? (3) o teste confere o **valor** ou só o status? (4) o nome diz a regra? (5) a inversão da regra deixa algum teste vermelho?

Continua sendo de vocês: o corte da história, a definição de pronto, o critério que o teste confere, a decisão do que o produto deve fazer em cada limite, a escolha da regra a inverter e a explicação de cada linha e de cada teste. Em qualquer nível, vocês continuam responsáveis por **verificar, testar, corrigir e defender** o resultado — e **"a IA escreveu assim" não é justificativa aceita**.